Session 01:

Tổng quan DevOps & Quy trình CI/CD

RIKKEI Academy®
where the dream begins

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

NỘI DUNG

1. Tổng quan DevOps và hạn chế của triển khai thủ công
2. Khái niệm CI/CD (quy trình build, test, deploy)
3. Mô hình môi trường Dev, Staging và Production
4. Kiến trúc triển khai hệ thống Microservices
5. Linux và vai trò triển khai
6. Quản lý quyền và các lệnh mạng

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKKEI Academy®
where the dream begins

1

Tổng quan DevOps và hạn chế của triển khai thủ công

1. Sự dịch chuyển tư duy của Kỹ sư phần mềm

Không ai trả tiền cho một ứng dụng chỉ chạy trên localhost. Đây là ranh giới giữa kỹ sư viết code và kỹ sư làm ra sản phẩm thật.

🖥️ Tư duy cũ — Local-centric

- Chỉ quan tâm viết code, nhấn "Run" trên IDE là xong việc
- Coi cấu hình mạng, cài OS, triển khai server là việc của người khác
- "Máy tôi chạy được mà!" — câu nói quen thuộc khi bị báo lỗi Production

🚀 Tư duy mới — Production-ready

- Code phải được đóng gói chuẩn hóa, tự động kiểm thử và tự động deploy
- Giám sát lỗi liên tục theo thời gian thực trên môi trường Production
- Chịu trách nhiệm về toàn bộ vòng đời vận hành của dòng code mình viết ra

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Các kịch bản “vỡ mồm” khi triển khai thủ công

Dự án QuickBite triển khai mỗi Module/Service trên 1 VPS cấu hình 1 vCPU / 1GB RAM. Đây là những gì thực sự xảy ra khi không có CI/CD.

1

Vòng lặp sửa lỗi vô tận

Sửa một dấu phẩy → scp → restart = **10 phút/lần**. Sửa 10 lỗi mất trọn một buổi sáng.

2

Environment Drift — Lệch cấu hình DB

Copy nhầm config local lên Prod. Dữ liệu thật bị ghi vào DB test. Rò rỉ thông tin.

3

Java Runtime Mismatch

Code Java 21, Server Java 17. Lỗi `UnsupportedClassVersionError` — sập app.

4

Hỗn loạn phiên bản file JAR

Server đầy file: `final.jar`, `v2.jar`... Không biết file nào an toàn để Rollback.

5

Log Blindness — Mù thông tin

App sập ngầm, SSH mò log thô hàng GB. Hệ thống chết, doanh nghiệp mất tiền theo giây.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

3. Bức tường đổ lỗi — Wall of Confusion

Quy trình bàn giao truyền thống tạo ra một khoảng trống trách nhiệm nguy hiểm giữa hai đội.



⚡ Đội Dev muốn

Cập nhật tính năng mới **thật nhanh** để kịp tiến độ. Mỗi thay đổi nhỏ đều muốn push ngay lên Production.

🛡️ Đội Ops muốn

Hệ thống **ổn định tuyệt đối**, ít thay đổi. Mỗi phiên bản mới là một rủi ro tiềm ẩn cần kiểm soát kỹ.

▶ **Kết quả:** Khi lỗi xảy ra, **Dev** đổ tại server — **Ops** đổ tại code. Không ai chịu trách nhiệm. Quy trình bàn giao đứt gãy hoàn toàn.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

4. Bản chất DevOps & 3 Trụ cột cho Kỹ sư Java

DevOps = Development + Operations — Không phải chức danh công việc, mà là sự giao thoa giữa **Văn hóa**, **Quy trình** và **Công cụ** để tự động hóa vòng đời phát hành phần mềm.

Am hiểu hệ thống thực tế

Hiểu về hardware (phần cứng) và infrastructure (hạ tầng) (CPU, RAM, SSD, Bandwidth, vv)

Hiểu sâu Java Runtime (JVM), hiểu về Spring

Biết cách Java hay framework quan trọng như Spring hoạt động như thế nào với hệ điều hành.

Xây CI/CD Pipeline

Sản phẩm cốt lõi của DevOps. Dây chuyền tự động hóa thay thế hoàn toàn việc gõ lệnh thủ công từng bước.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKKEI Academy®
where the dream begins

2

Khái niệm CI/CD (quy trình build, test, deploy)

1. Vấn đề thực tế & Định nghĩa CI vs CD

Kịch bản quen thuộc: Tech Lead mất hàng giờ mỗi ngày chỉ để build và test thủ công code của Intern trước khi dám merge. Anh ấy đang làm việc của một "intern cao cấp".

CI — Continuous Integration

Tự động **Compile** và chạy **Unit Test** ngay khi dev gõ **git push**. Triết lý **Fail-fast**: phát hiện và chặn lỗi logic ngay lập tức.

Continuous Delivery

Sau CI thành công, code tự động đóng gói sẵn sàng deploy. Cần một thao tác **bấm nút duyệt thủ công** để lên Production.

Continuous Deployment

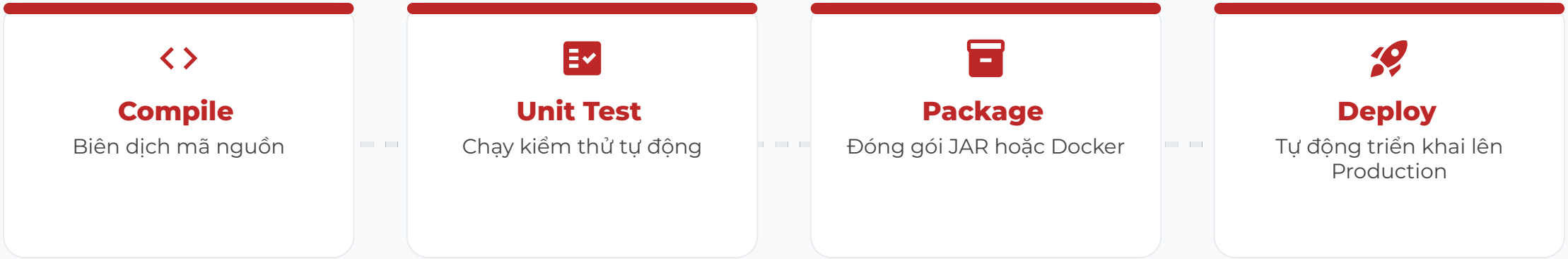
Tự động hóa **100%**. Code qua CI sẽ tự chạy script deploy thẳng lên Production — không cần phê duyệt của con người.

Git Webhook là chìa khóa: Ngay khi intern gõ **git push**, hệ thống tự động kích hoạt máy chủ CI chạy build và test. Báo đỏ thì intern tự sửa — Tech Lead chỉ review code xanh.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Sơ đồ Pipeline CI/CD tiêu chuẩn

Bất kỳ stage nào thất bại, toàn bộ pipeline **dừng ngay lập tức** để bảo vệ server Production khỏi code lỗi.



Hiểu lầm kinh điển: Viết **script** copy file JAR lên server tự động **KHÔNG phải là CI/CD**. Thiếu hoàn toàn cấu phần CI — không có Unit Test kiểm soát chất lượng.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKKEI Academy®
where the dream begins

3

Mô hình môi trường Dev, Staging và Production

1. Bản đồ 3 Môi trường — Kiểm soát rủi ro phát hành

Dự án QuickBite vận hành 3 môi trường hoàn toàn độc lập. Mỗi môi trường có vai trò và mức độ bảo mật khác nhau rõ ràng.

🛠️ Dev (Development)

Máy local của dev. Dữ liệu giả lập tự tạo, yêu cầu ổn định thấp. Mục tiêu duy nhất: viết code nhanh và debug trực tiếp.

🧪 Staging (Pre-prod)

VPS độc lập mô phỏng giống Prod 99%. Dữ liệu mô phỏng thật nhưng không nhạy cảm. Chạy Integration Test và UAT nghiệm thu tính năng.

🚀 Production (Prod)

Chạy thật phục vụ người dùng cuối. Dữ liệu thật và nhạy cảm. Yêu cầu bảo mật tuyệt đối và uptime **99.99%**.

Nguyên tắc "Build Once, Run Anywhere": Chỉ biên dịch ra một file JAR duy nhất trên CI server. Dùng chính file JAR đó chạy trên cả 3 môi trường. Điểm khác biệt duy nhất: các biến cấu hình nạp vào lúc khởi chạy.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Tách biệt cấu hình bằng Biến môi trường

Nguy hiểm thực tế: Hardcode mật khẩu database trong **application.yml** — dù là Private Repo vẫn có hàng chục dev/intern đọc được. Đây là lỗ hổng bảo mật nghiêm trọng.

```
spring:
  datasource:
    url:
${QUICKBITE_DB_URL:jdbc:postgresql://localhost:5432/qu
ickbite_user}
    username: ${QUICKBITE_DB_USER:postgres}
    password: ${QUICKBITE_DB_PASS:password123}
```

Spring Boot ưu tiên biến môi trường hệ thống. Giá trị sau dấu : chỉ dùng làm mặc định tại local.

```
# 1. Nạp các cấu hình của môi trường Staging vào hệ
điều hành
export
QUICKBITE_DB_URL=jdbc:postgresql://10.0.1.20:5432/quic
kbite_user_staging
export QUICKBITE_DB_USER=staging_admin
export QUICKBITE_DB_PASS=StagingSecurePass@2026

# 2. Khởi chạy file JAR duy nhất
java -jar user-service-0.0.1.jar
```

"Build Once, Run Anywhere": Một file JAR duy nhất cho mọi môi trường, chỉ thay đổi cấu hình đầu vào.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKKEI Academy®
where the dream begins

4

Kiến trúc triển khai hệ thống Microservices

1. Kiến trúc triển khai hệ thống Microservices

Client gọi thẳng IP nội bộ từng service (`http://10.0.1.15:8081`) là một sai lầm kiến trúc với 3 hậu quả:

🔓 Lộ cổng nội bộ

Phơi các port nội bộ ra internet.

Hacker dễ dàng quét cổng (*port scanning*) và tấn công trực tiếp từng service.

🔒 SSL phức tạp

Phải quản lý chứng chỉ HTTPS riêng cho từng cổng của từng service.

Chi phí vận hành tăng theo cấp số nhân.

🔗 Cứng nhắc IP

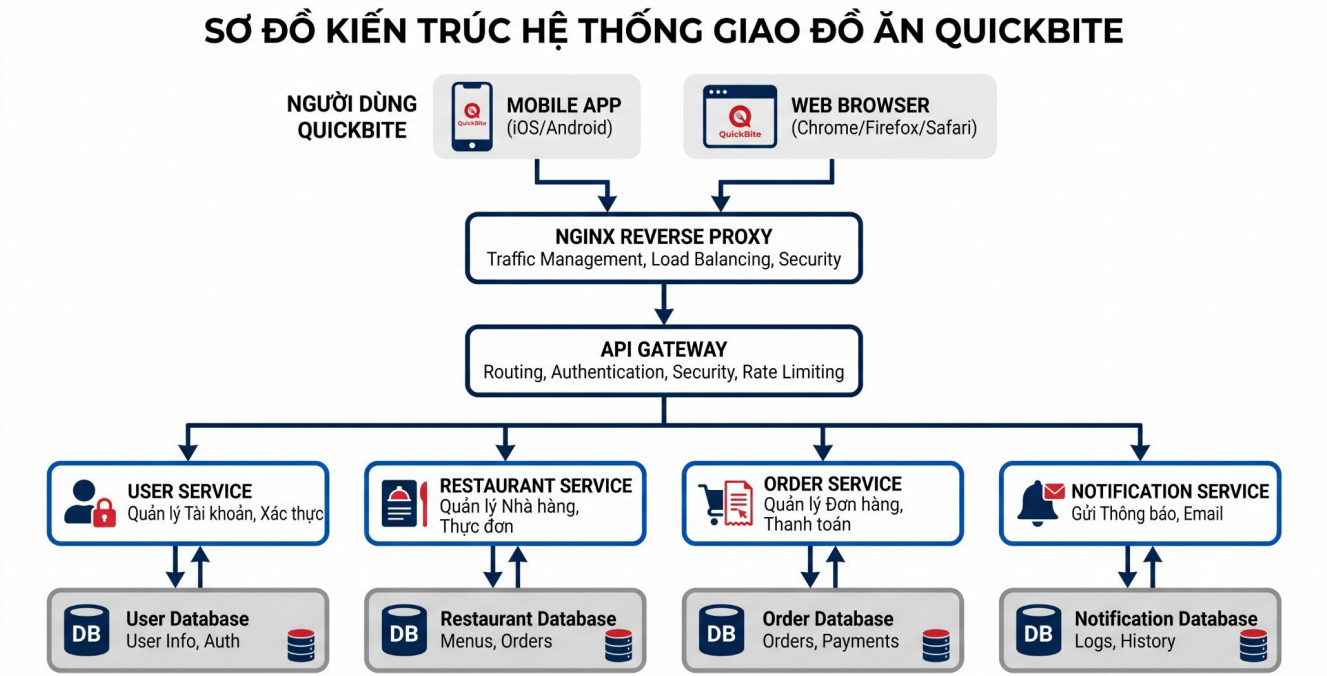
Khi đổi IP server hoặc scale, phải cập nhật tay tất cả client.

Hệ thống mất tính linh hoạt hoàn toàn.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

1. Kiến trúc phân lớp QuickBite — Sơ đồ luồng dữ liệu

Hệ thống QuickBite sử dụng mô hình phân lớp bảo mật nghiêm ngặt: chỉ một điểm vào duy nhất từ internet, toàn bộ service nằm trong mạng nội bộ.



Nguyên tắc Database-per-service: Mỗi service chỉ truy cập database riêng của mình. Vật lý có thể chung một cụm PostgreSQL để tiết kiệm chi phí, nhưng logic phải tách biệt tuyệt đối — không service nào được phép đọc/ghi vào database của service khác.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Vai trò chi tiết: Nginx & API Gateway

🛡️ Nginx — Edge Layer

Tấm khiên ngoài cùng. Tiếp nhận kết nối qua cổng tiêu chuẩn 80/443.

- Trả file tĩnh Frontend nhanh (**web.quickbite.com**)
- Giải mã SSL (SSL Termination) cho toàn hệ thống một lần
- Forward request API (**api.quickbite.com**) vào API Gateway

🔀 API Gateway

Đứng sau Nginx, tiếp nhận request nội bộ và điều phối toàn bộ traffic.

- Định tuyến động theo Path: **/api/v1/users** → **user-service**
- Xác thực tập trung (Authentication) cho tất cả service
- Rate Limiting — giới hạn lượt gọi chống DDoS

Kết quả: Toàn bộ lớp dịch vụ nội bộ **ẩn hoàn toàn** sau Nginx và API Gateway. Hacker từ internet không thể quét hoặc tấn công trực tiếp vào bất kỳ microservice nào.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

5

Linux và vai trò triển khai

1. Tại sao Linux là chuẩn mực của DevOps?

Linux gọn nhẹ, bảo mật cao, vận hành ổn định qua CLI. Quan trọng hơn: công nghệ Container (Docker) bắt buộc dựa trên **Linux Kernel** thông qua cơ chế *Shared Kernel*.

🛠️ Khởi tạo VPS mới

Lệnh bắt buộc đầu tiên:

```
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y openjdk-17-jdk curl git
```

Cập nhật toàn bộ bản vá bảo mật và cài JDK 17 để chạy Spring Boot. **Không bỏ qua bước này.**

⚡ Tự động hóa với Script

initial-script.sh:

```
#!/bin/bash
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y openjdk-17-jdk curl git
```

Cấp quyền `chmod +x` rồi chạy. Mỗi VPS mới chạy đúng một lần, đảm bảo **đồng nhất môi trường**.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Lệnh Linux thực chiến

Linux gọn nhẹ, bảo mật cao. **Linux Kernel** là nền tảng bắt buộc của Docker thông qua cơ chế *Shared Kernel*.

📁 Thao tác file & thư mục

- `pwd` — Xem đường dẫn hiện tại
- `ls -la` — Liệt kê toàn bộ file (cả ẩn)
- `cd [path]` — Di chuyển (.. lùi về cha)
- `mkdir -p` — Tạo cây thư mục cha
- `cat` / `nano` — Đọc & Soạn thảo trực tiếp
- `cp` / `mv` / `rm -rf` — Sao chép, Di chuyển, Xóa

⚙️ Quản lý tiến trình & Dịch vụ

Tiến trình Java:

ps -ef | grep java: Tìm PID đang chạy ngầm.
kill -9 [PID]: Dừng khẩn cấp khi bị treo.

🔄 **Systemd (Dịch vụ):**

sudo systemctl start/stop/status service
sudo systemctl enable service

enable: Tự khởi động cùng OS. **Cực kỳ quan trọng cho Production.**

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

3. Biến môi trường lâu dài & Điều khiển Nginx

🔧 Biến môi trường vĩnh viễn

```
echo "export QUICKBITE_DB_USER=staging_admin"  >> ~/.bashrc
source ~/.bashrc
```

Ghi vào **.bashrc** để biến tồn tại vĩnh viễn qua mọi phiên đăng nhập.

source nạp cấu hình ngay lập tức mà không cần logout hệ thống.

🌐 Điều khiển Nginx an toàn

```
sudo nginx -t
```

Quan trọng: Kiểm tra lỗi cú pháp file cấu hình trước khi áp dụng thực tế.

```
sudo nginx -s reload
```

Tải lại cấu hình **không ngắt kết nối** khách hàng.

Ưu tiên dùng **reload** thay vì restart trên môi trường Production.

Các lệnh Docker và Docker compose: sẽ được nhắc đến kĩ hơn trong từng bài tiếp theo, không cần liệt kê ở đây

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKKEI Academy®
where the dream begins

6

Quản lý quyền và các lệnh mạng

1. Nguy hiểm khi chạy App bằng quyền Root

"Không bao giờ chạy ứng dụng Java bằng user root"

User root có quyền tối cao can thiệp vào hệ điều hành. Nếu hacker khai thác lỗ hổng code Java, họ sẽ chiếm quyền root — xóa sạch dữ liệu hoặc chiếm luôn máy chủ vật lý.

🛡️ Giải pháp: Tạo user hệ thống chuyên dụng

```
sudo mkdir -p /opt/quickbite/user-service
sudo chown -R quickbite:quickbite /opt/quickbite
sudo chmod -R 750 /opt/quickbite
```

-r: Tài khoản hệ thống, không tạo thư mục home.
-s /bin/false: Vô hiệu hóa login shell — ngăn chặn SSH vào server.

7 — rwx (Owner)

User **quickbite**: Đầy đủ quyền **Đọc, Ghi, Thực thi**. Cần thiết để chạy và ghi log.

5 — r-x (Group)

Group **quickbite**: Quyền **Đọc và Thực thi**. Đủ để khởi chạy service, không thể sửa file.

0 — --- (Others)

Người dùng khác: **Không có bất kỳ quyền nào**. Bảo vệ tuyệt đối thông tin cấu hình nhạy cảm.

Cấp quyền chạy build script Gradle: `chmod +x gradlew`

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

2. Chẩn đoán lỗi mạng & Xử lý xung đột cổng

🔍 Lệnh chẩn đoán mạng cơ bản

`ip addr`
Kiểm tra địa chỉ IP hiện tại của server Linux

`ping 10.0.1.20`
Kiểm tra kết nối vật lý đến máy chủ database

`curl -I https://api.quickbite.com`
Kiểm tra nhanh backend có phản hồi HTTP không

🚨 Xử lý cổng 8080 đang bị chiếm

Spring Boot báo: "Web server failed to start. Port 8080 was already in use."

```
sudo ss -tulpn | grep :8080
```

Kết quả: tcp LISTEN *:8080 users:(("java",pid=14522))

```
sudo kill -9 14522
```

✅ **Giải phóng cổng, khởi động lại service bình thường.**

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

TỔNG KẾT

- Tổng quan hệ thống Quickbite
- Phân biệt các môi trường Dev / Staging / Production
- Tầng kiến trúc Quickbite: Client -> Nginx -> Gateway
- Các lệnh linux cơ bản

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

KẾT THÚC

HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN